Programación Orientada a Objetos en C#

Ruta de estudio

1. Herencia
2. Abstracción y polimorfismo
3. Clases abstractas e interfaces
4. Diagramas UML: de clases y de secuencia
5. Composición, estructura de una clase y excepciones
6. Pruebas unitarias en C#
7. Serialización y DTO
8. Cola de prioridad y `await`
9. Resumen y preguntas de repaso

Cómo estudiar con estos apuntes. Lee cada sección, escribe los ejemplos en tu editor y, al final, responde las preguntas de repaso sin mirar las respuestas.

1. Herencia

La herencia permite que una clase hija reciba los atributos y métodos de una clase padre sin escribirlos de nuevo; la hija solo agrega lo que la hace especial.

1.1 Qué es

- Es heredar atributos y métodos de otra clase, sin volver a escribirla.
- Va de la **clase padre** (general) a la **clase hija** (subtipo o especialización).
- La clase padre se define como una clase normal, con lo básico; lo que interesa es crear los subtipos, los hijos.
- En la hija se indica el nombre de la clase padre.
- Por defecto, la hija hereda los comportamientos del padre.

1.2 La regla «es un»

Para decidir si una clase debe ser padre de otra, lee la relación en voz alta: si suena natural, se escoge.

Ejemplo de apoyo: «un Gerente **es un** Empleado» suena natural, así que `Empleado` es el padre y `Gerente` la hija. «Un Empleado es un Gerente» no suena natural, por eso no se hace al revés.

1.3 El constructor de la clase hija

El constructor de la hija recibe los atributos del padre **y** los suyos, pero solo se ocupa de validar los suyos. De los demás se encarga el padre.

- En **Python** se llama al constructor del padre con `super()`.
- En **C#** se usa `: base(...)` después de la firma del constructor.
- Al crear el objeto solo se definen los atributos que le importan al subtipo.
- Recuerda definir el constructor con parámetros para que no se pueda crear un empleado vacío.

Ejemplo de apoyo en C#:

```
public class Empleado
{
    public string Nombre { get; protected set; }
    public decimal Salario { get; protected set; }

    public Empleado(string nombre, decimal salario)
    {
        if (string.IsNullOrWhiteSpace(nombre))
            throw new ArgumentException("El nombre es obligatorio");
        Nombre = nombre;
        Salario = salario;
    }
}

public class Gerente : Empleado
{
    public decimal Bono { get; private set; }

    public Gerente(string nombre, decimal salario, decimal bono)
        : base(nombre, salario)      // el padre valida y guarda lo suyo
    {
        if (bono < 0)
            throw new ArgumentException("El bono no puede ser negativo");
        Bono = bono;                 // la hija solo valida lo suyo
    }
}

// crear el objeto
var gerente = new Gerente("Laura", 4000000m, 500000m);
```

Ejemplo de apoyo en Python:

```
class Empleado:
    def __init__(self, nombre, salario):
        self.nombre = nombre
        self.salario = salario

class Gerente(Empleado):
    def __init__(self, nombre, salario, bono):
        super().__init__(nombre, salario)   # el padre se encarga de los
suyos
        if bono < 0:
            raise ValueError("El bono no puede ser negativo")
        self.bono = bono
```

1.4 `protected` y el encapsulamiento en C#

Si un atributo del padre se marca como `protected`, queda escondido para el resto del programa y solo se puede modificar desde la propia clase y sus hijos. `protected` es el modificador que define el encapsulamiento en la herencia.

Modificador	Quién puede usarlo
`public`	Cualquier parte del programa
`protected`	La clase y sus hijas
`private`	Solo la propia clase (las hijas tampoco)

Por eso, en las propiedades del padre se escribe `protected set` y no `private set`: con `private`, las clases hijas no podrían asignar el valor.

2. Abstracción y polimorfismo

La abstracción pone lo común en una clase general; el polimorfismo deja que cada clase hija cambie cómo realiza una acción que se llama igual en todas.

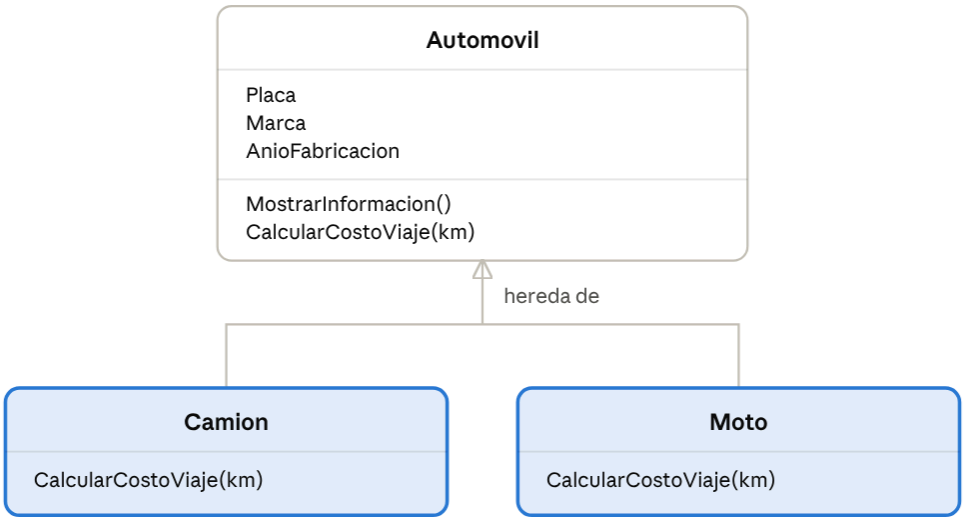
2.1 Abstracción: el ejemplo de los automóviles

La clase principal `Automovil` guarda lo que tienen en común todos los vehículos:

- Atributos: placa, marca y año de fabricación.
- Un método central para mostrar la información.

De ella salen dos clases hijas, `Camion` y `Moto`. La diferencia entre ellas es **cómo calculan el costo del viaje**.

Camion y Moto heredan de Automovil y cambian cómo calculan el costo



Lo único que cambia en las hijas es CalcularCostoViaje(km); lo demás lo heredan del padre.

diagrama de clases · Automovil y sus dos hijas

La flecha con punta hueca va de cada hija hacia el padre. `Camion` y `Moto` heredan placa, marca, año y `MostrarInformacion()`, y solo reescriben `CalcularCostoViaje(km)`.

2.2 Polimorfismo

Es el comportamiento particular sobre algo general: permite trabajar con los métodos de los hijos desde la clase padre, como si fuera el `main`.

- En la **clase padre** se define la firma del método (cómo se llama y qué recibe).
- En la **clase hija** se escribe el procedimiento (qué hace).

	C#	Python
En el padre	`public virtual`: indica que el método se puede sobrescribir	Se declara el método con `pass`
En el hijo	`override`: sobrescribe el método del padre	Se define el método con el mismo nombre
Si el hijo no lo define	Toma el método del padre	Toma el del padre (que solo tiene `pass`)
Cómo se comprueba	Al compilar	Con *duck typing*: falla en tiempo de ejecución

Duck typing quiere decir que Python no revisa el tipo del objeto: si tiene el método, lo llama; si no lo tiene, el error aparece cuando se ejecuta el programa.

Ejemplo de apoyo en C# (las tarifas son inventadas, solo para ilustrar):

```
public class Automovil
{
    public string Placa { get; protected set; }
    public string Marca { get; protected set; }
    public int AnioFabricacion { get; protected set; }

    public Automovil(string placa, string marca, int anioFabricacion)
    {
        Placa = placa;
        Marca = marca;
        AnioFabricacion = anioFabricacion;
    }

    public void MostrarInformacion()
        => Console.WriteLine($"{Placa} - {Marca} ({AnioFabricacion})");

    // la firma se define en el padre; virtual permite sobrescribirla
    public virtual decimal CalcularCostoViaje(decimal km) => 0;
}

public class Camion : Automovil
{
    public Camion(string placa, string marca, int anio) : base(placa, marca,
anio) { }

    public override decimal CalcularCostoViaje(decimal km) => km * 5000m;
}

public class Moto : Automovil
{
    public Moto(string placa, string marca, int anio) : base(placa, marca,
anio) { }

    public override decimal CalcularCostoViaje(decimal km) => km * 1500m;
}

// el polimorfismo: una lista del tipo general, cada objeto calcula a su
manera
var vehiculos = new List<Automovil>
{
    new Camion("ABC123", "Volvo", 2020),
    new Moto("XYZ98A", "Yamaha", 2022)
};
```

```
foreach (Automovil v in vehiculos)
{
    v.MostrarInformacion();
    Console.WriteLine(v.CalcularCostoViaje(100));
}
```

Ejemplo de apoyo en Python:

```
class Automovil:
    def calcular_costo_viaje(self, km):
        pass                        # cada hija lo define

class Camion(Automovil):
    def calcular_costo_viaje(self, km):
        return km * 5000

class Moto(Automovil):
    def calcular_costo_viaje(self, km):
        return km * 1500

for vehiculo in [Camion(), Moto()]:
    print(vehiculo.calcular_costo_viaje(100))
```

3. Clases abstractas e interfaces

Una clase abstracta no se puede instanciar y obliga a sus hijas a implementar ciertos comportamientos; una interfaz es un contrato que puede cumplir cualquier clase, esté o no emparentada con otra.

3.1 ¿Hay que poder instanciar al padre?

Depende de lo que se necesite.

- **Clase normal:** se puede instanciar tanto el padre como el hijo.
- **Clase abstracta:** el padre no se debe instanciar. Sirve para obligar a que las hijas usen los comportamientos definidos en el padre, sobrescribiendo el método que está en la clase abstracta.

3.2 Interfaces

Una interfaz genera un **contrato**: por obligación, toda clase que la implemente debe hacer tal cosa.

- Solo declara métodos; todos son abstractos.

- Es una forma de decir «todas las clases deben implementar esto», y cada una lo resuelve como pueda.
- El contrato también se puede poner en la clase padre: así todas las hijas lo cumplen, cada una a su manera, sobrescribiendo pero respetando el contrato.

3.3 Diferencias

	Clase abstracta	Interfaz
Qué es	Un padre que no se instancia y obliga a las hijas a sobrescribir	Un contrato: solo define qué se debe hacer
Relación entre clases	Las hijas están relacionadas por herencia	No necesita ninguna relación entre las clases
Cuántas se pueden usar	Se hereda de **una sola**	Se pueden implementar **todas las que se quiera**

Ejemplo de apoyo en C#:

```
// clase abstracta: no se puede hacer new Empleado(...)
public abstract class Empleado
{
    public string Nombre { get; protected set; }

    protected Empleado(string nombre) { Nombre = nombre; }

    // las hijas están obligadas a implementarlo
    public abstract decimal CalcularPago();
}

public class EmpleadoFijo : Empleado
{
    public decimal Salario { get; private set; }

    public EmpleadoFijo(string nombre, decimal salario) : base(nombre)
    {
        Salario = salario;
    }

    public override decimal CalcularPago() => Salario;
}

// interfaces: solo el contrato (por convención, el nombre empieza con I)
public interface IImprimible { void Imprimir(); }
```

```
public interface IExportable { void Exportar(); }

// hereda de una sola clase, pero implementa todas las interfaces que quiera
public class Contratista : Empleado, IImprimible, IExportable
{
    public Contratista(string nombre) : base(nombre) { }

    public override decimal CalcularPago() => 0;
    public void Imprimir() { Console.WriteLine(Nombre); }
    public void Exportar() { /* guardar en un archivo */ }
}
```

4. Diagramas UML: de clases y de secuencia

UML es un lenguaje gráfico estándar para representar el diseño de un sistema; no depende de la sintaxis de ningún lenguaje de programación.

4.1 Para qué sirve

Es un tema **agnóstico**: no está relacionado con la sintaxis. Se usa en dos momentos:

- **Al diseñar:** para comunicar el diseño antes de programarlo.
- **Para documentar:** para dejar registrado lo que ya existe.

4.2 Diagrama de clases

Muestra las clases y cómo se relacionan.

- **Asociaciones simples y compuestas:** son las conexiones entre clases.
- **Multiplicidad:** indica cómo varía la conexión, es decir, cuántas veces se va a instanciar una clase por cada instancia de la otra.
- **Clases abstractas e interfaces:** se señalan escribiendo el nombre en *cursiva* o con la etiqueta `<<abstract>>` o `<<interface>>` sobre el nombre.

Multiplicidad	Se lee	Cómo se escribe en el extremo de la línea
De 1 a 1	Cada objeto se conecta con exactamente uno	`1` en ambos extremos
De 1 a varios	Un objeto se conecta con muchos	`1` en un extremo y `*` en el otro
De 1 a otro número	Un objeto se conecta con una cantidad fija	`1` en un extremo y el número (por ejemplo `3`) en el otro

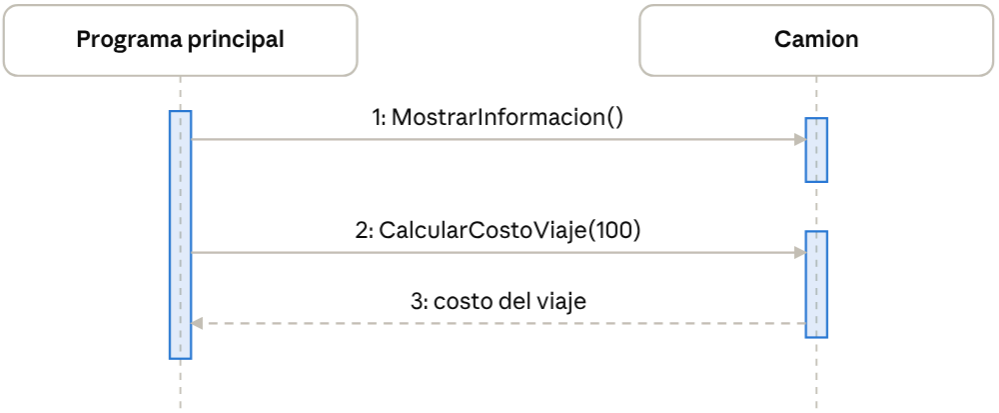
4.3 Diagrama de secuencia

Muestra **cómo interactúan** los elementos del programa, por procesos y paso a paso.

- **Flechas:** una indica que se llama algo de otra clase (una acción), y otra, que se recibe algo de otra clase.
- **Barra de activación:** es la barra que se dibuja sobre cada clase; marca los periodos en que la clase está haciendo algo que se le solicitó, es decir, que se está usando.
- **Orden con índice:** una clase puede pedir una rutina y, mientras espera su resultado, pedir otras cosas o recibir otras. Por eso se numera cada mensaje: el índice indica el orden de pedir y recibir.

Ejemplo de apoyo, con las clases de la sección 2:

El programa principal llama al Camion dos veces y recibe el costo



Flecha continua: llamada. Flecha punteada: respuesta. Barra: la clase está haciendo algo que se le pidió.

diagrama de secuencia de apoyo · 3 mensajes

Los mensajes se numeran para dar el orden: primero se pide la información, luego se pide el costo del viaje y el `Camion` responde con el resultado.

5. Composición, estructura de una clase y excepciones

En la composición una clase se construye con objetos de otras clases; para escribirla ordenada se sigue un orden fijo de miembros, y las excepciones permiten detectar errores y que el programa siga funcionando.

5.1 Composición

Una clase puede estar compuesta por **una o varias clases**: en lugar de heredar, guarda objetos de otras clases como atributos.

Ejemplo: un `Empleado` está compuesto por una lista de objetos `Pago`.

5.2 Estructura recomendada de una clase

Se propone escribir los miembros de la clase en este orden:

1. **Constantes**
2. **Atributos que se privatizan** (los campos privados)
3. **Atributos**
4. **Encapsulamiento** (las propiedades con `get` y `set`)

Para convertir texto, `value.ToUpper()` pasa un texto a mayúsculas y `value.ToLower()` a minúsculas. Se usan dentro del `set`, donde `value` es el valor que se está asignando.

5.3 Un atributo se puede asignar con un método

Si un atributo depende de una condición (por ejemplo, de cómo es la carga), se calcula con un método y el atributo se asigna con el resultado de ese método.

Ejemplo de apoyo en C# (el límite de 10 toneladas es inventado, solo para ilustrar):

```
public class Pago
{
    public decimal Monto { get; private set; }

    public Pago(decimal monto) { Monto = monto; }
}

public class Empleado
{
    // 1. constantes
    private const decimal LimiteBonoPorPago = 5000000m;

    // 2. atributos que se privatizan
    private string nombre;
    private List<Pago> pagos = new List<Pago>();

    // 3. atributos
    public int Antiguedad { get; private set; }

    // 4. encapsulamiento
    public string Nombre
    {
        get => nombre;
        set => nombre = value.ToUpper();   // siempre se guarda en mayúsculas
    }

    public Empleado(string nombre, int antiguedad)
```

```
    {
        Nombre = nombre;
        Antiguedad = antiguedad;
    }

    public void AgregarPago(Pago pago) => pagos.Add(pago);   // composición
}

public class Envio
{
    public decimal Carga { get; private set; }
    public string TipoCarga { get; private set; }

    public Envio(decimal carga)
    {
        Carga = carga;
        TipoCarga = ClasificarCarga();     // el atributo se asigna con un
método
    }

    private string ClasificarCarga() => Carga > 10 ? "Pesada" : "Liviana";
}
```

5.4 Obtener el mejor pago: recorrer una lista

Es el mismo patrón de «mayor y menor» de Fundamentos de Programación: se asume que el mejor pago es el que está en la primera posición `[0]` y después, con un `foreach`, se recorre toda la lista comparando.

```
// dentro de la clase Empleado
public Pago ObtenerMejorPago()
{
    if (pagos.Count == 0)
        throw new InvalidOperationException("El empleado no tiene pagos");

    Pago mejor = pagos[0];                // se asume que el mejor es el
primero
    foreach (Pago pago in pagos)
    {
        if (pago.Monto > mejor.Monto)
            mejor = pago;
    }
    return mejor;
}
```

`Any` es un método de LINQ para preguntar por los elementos de una colección:

```
bool hayPagos = pagos.Any();                // true si hay al menos un pago
bool sinPagos = !pagos.Any();               // true si no hay ninguno
bool hayGrandes = pagos.Any(p => p.Monto > 1000000m);   // true si alguno
cumple
```

5.5 Excepciones y depuración

Término	Qué es
Excepción	Un error que el programa lanza para que se pueda reconocer fácilmente dónde falló el código
Debug (depurar)	El proceso de identificar, analizar y corregir errores en el código de un programa o sistema para asegurar su correcto funcionamiento
`try/catch`	Estructura que captura una excepción para que el programa siga funcionando

```
try
{
    var empleado = new Empleado("Laura", 3);
    var mejor = empleado.ObtenerMejorPago();
}
catch (InvalidOperationException ex)
{
    Console.WriteLine($"Error: {ex.Message}");   // el programa continúa
}
```

6. Pruebas unitarias en C#

Una prueba unitaria comprueba **una sola cosa** (un método, una función o una clase), de forma aislada, y siempre sigue tres pasos: Arrange, Act y Assert.

6.1 Qué es una prueba unitaria

- No es una prueba de un sistema completo: es una prueba a una sola cosa, ya sea un método, una función o una clase.
- Es **aislada**: no necesita ninguna dependencia. Se trata de probar que cada partecita funcione por separado.
- Sirve para asegurarnos de que la información sea la esperada; por ejemplo, que el objeto se cree como debería ser, según como lo definimos.

6.2 Estructura de la prueba: Arrange, Act, Assert

Paso	Qué se hace
Arrange (preparar)	Preparar los datos y los objetos que se necesitan para la prueba
Act (actuar)	Ejecutar lo que se está probando (llamar al método)
Assert (verificar)	Comprobar que el resultado es el esperado

6.3 `[Fact]` y `[Theory]` en C#

Atributo	Cuándo usarlo
`[Fact]`	Para probar **una sola cosa**, con un solo caso
`[Theory]`	Para probar **varias cosas**: se le pasan diferentes datos a la misma prueba

Ejemplo de apoyo con xUnit, usando las clases de las secciones anteriores:

```
using Xunit;

public class PruebasDelEjemplo
{
    [Fact]
    public void Constructor_ConDatosValidos_CreaElObjetoComoSeDefinio()
    {
        // Arrange: preparar los datos
        string nombre = "Laura";
        int antiguedad = 3;

        // Act: ejecutar lo que se prueba
        var empleado = new Empleado(nombre, antiguedad);

        // Assert: verificar el resultado esperado
        Assert.Equal("LAURA", empleado.Nombre);   // el set lo guarda en
mayúsculas
        Assert.Equal(3, empleado.Antiguedad);
    }

    [Theory]
    [InlineData(100, 500000)]    // km, costo esperado
```

```
    [InlineData(10, 50000)]
    [InlineData(0, 0)]
    public void CalcularCostoViaje_DelCamion_MultiplicaPorLaTarifa(decimal
km, decimal esperado)
    {
        // Arrange
        var camion = new Camion("ABC123", "Volvo", 2020);

        // Act
        decimal costo = camion.CalcularCostoViaje(km);

        // Assert
        Assert.Equal(esperado, costo);
    }
}
```

6.4 Detalles que conviene recordar

- **Clases abstractas:** no se pueden instanciar, así que se prueban a través de sus **clases hijas**.
- **Ejecutar:** con *Run test* en el IDE se corre la prueba y se ve si pasó o falló.
- **Python:** también se pueden escribir pruebas unitarias. En PyCharm (de JetBrains) el IDE muestra los resultados de forma gráfica, igual que en C#.

7. Serialización y DTO

La serialización convierte un objeto en un formato que se puede guardar o enviar (por ejemplo JSON), y el DTO es la clase sencilla que se usa para transportar esos datos sin exponer la entidad del dominio.

7.1 Serialización

Es el proceso de pasar objetos y sus datos a un formato que pueda almacenarse o transmitirse. Al leer un documento, el formato en que está escrito determina cómo se debe convertir de vuelta a un objeto.

7.2 DTO (Data Transfer Object)

DTO significa **objeto de transferencia de datos**.

- Es una clase en la que **todo es público**.
- **No valida y no decide**: solo lleva datos de un lado a otro.

- La **entidad** del dominio, en cambio, es la experta: guarda las reglas y es un proceso más privado.

	Entidad del dominio	DTO
Visibilidad	Atributos protegidos, con encapsulamiento	Todo público
Valida datos	Sí	No
Para qué sirve	Aplicar las reglas del negocio	Transportar datos (guardar, enviar, leer)

7.3 Mapeo

Mapear es **convertir de un objeto a otro**: pasar lo que se tiene en el dominio a lo que se quiere hacer con ayuda de un DTO (y al revés). El método de conversión se crea en la clase de la que se quiere partir, agregándole la firma del DTO.

7.4 JSON: nombres, `null` e ignorar campos

Para pasar el DTO a JSON se usa una biblioteca (por ejemplo, `System.Text.Json`).

Necesidad	Cómo se resuelve
El nombre de la variable es distinto al del JSON	Se indica con un atributo: «esta variable corresponde a este nombre» (`[JsonPropertyName("...")]`)
El valor puede venir `null`	Se declara el tipo con `?`, por ejemplo `double?`: puede ser un número o `null`
Un campo no debe aparecer en el JSON	Se marca con `[JsonIgnore]`: le dice «ignora esta variable» y no se muestra

Ejemplo de apoyo en C#:

```
using System.Text.Json;
using System.Text.Json.Serialization;

// DTO: todo público, no valida ni decide
public class EmpleadoDto
{
    [JsonPropertyName("nombre_completo")]   // el nombre en el JSON es
distinto
```

```
    public string Nombre { get; set; }

    public double? Salario { get; set; }    // puede venir null

    [JsonIgnore]                            // no aparece en el JSON
    public string NotaInterna { get; set; }
}

// Entidad del dominio: valida y protege sus datos
public class Empleado
{
    public string Nombre { get; private set; }
    public double Salario { get; private set; }

    public Empleado(string nombre, double salario)
    {
        if (string.IsNullOrWhiteSpace(nombre))
            throw new ArgumentException("El nombre es obligatorio");
        if (salario < 0)
            throw new ArgumentException("El salario no puede ser negativo");
        Nombre = nombre;
        Salario = salario;
    }

    // mapeo: de la entidad al DTO
    public EmpleadoDto ToDto() => new EmpleadoDto { Nombre = Nombre, Salario
= Salario };
}

// serializar: objeto -> JSON
var empleado = new Empleado("Laura", 2500000);
string json = JsonSerializer.Serialize(empleado.ToDto());

// deserializar: JSON -> DTO
EmpleadoDto leido = JsonSerializer.Deserialize<EmpleadoDto>(json);
```

7.5 Cómo tratar un valor que puede ser `null`

Cuando una variable del DTO puede ser `double` o `null`, evita ser optimista con `!` y `.Value`: eso le dice al programa «esto siempre va a devolver un valor», y si no es así, falla.

```
// optimista: asume que siempre hay valor; si es null, el programa se cae
double salario = dto.Salario!.Value;

// mejor: revisar la variable del DTO que puede traer el problema
if (!dto.Salario.HasValue)
    throw new ArgumentException("El salario es obligatorio");
double salarioSeguro = dto.Salario.Value;
```

9. Resumen y preguntas de repaso

Si puedes explicar cada fila de la tabla siguiente con tus palabras y escribir un ejemplo corto, dominas el contenido de estos apuntes.

9.1 Resumen de conceptos

Concepto	Idea clave	En C#
Herencia	La hija recibe atributos y métodos del padre; se escoge con la regla «es un»	`class Gerente : Empleado`
Constructor de la hija	Valida solo lo suyo; el padre se encarga de lo del padre	`: base(...)` (en Python, `super()`)
`protected`	Visible para la clase y sus hijas, escondido para el resto	`protected set;`
Abstracción	Poner lo común en una clase general	`Automovil` con placa, marca y año
Polimorfismo	Un mismo método, comportamiento distinto en cada hija	`virtual` en el padre, `override` en la hija
Clase abstracta	Padre que no se instancia y obliga a las hijas a sobrescribir	`abstract`
Interfaz	Contrato sin relación de herencia; se pueden implementar varias	`interface IImprimible`
Composición	Una clase compuesta por objetos de una o varias clases	`List<Pago>` dentro de `Empleado`

Concepto	Idea clave	En C#
UML	Lenguaje gráfico estándar para diseñar y documentar; no depende del lenguaje	Diagramas de clases y de secuencia
Prueba unitaria	Prueba aislada de una sola cosa, con Arrange, Act y Assert	`[Fact]` y `[Theory]`
Serialización	Pasar un objeto a un formato que se guarda o se envía	`JsonSerializer.Serialize(...)`
DTO	Clase pública que solo transporta datos: no valida ni decide	`EmpleadoDto`
Cola de prioridad	Primero la prioridad, luego el orden de llegada	`PriorityQueue<TElement, TPriority>`

9.2 Preguntas de repaso

Responde primero sin mirar la columna de la derecha.

Pregunta	Respuesta
¿Cómo decides si una clase debe ser padre de otra?	Con la regla «es un»: se lee la relación y, si suena natural, se escoge.
¿Qué valida el constructor de una clase hija?	Solo los atributos propios; los del padre los valida el padre, al que se llama con `: base(...)` en C# o `super()` en Python.
¿Qué diferencia hay entre `protected` y `private`?	`protected` lo pueden usar la clase y sus hijas; `private`, solo la propia clase.
¿Qué es el polimorfismo y cómo se hace en C#?	Cada hija da su propio comportamiento a un método general: `virtual` en el padre y `override` en las hijas.
¿Qué pasa si la hija no sobrescribe un método `virtual`?	Usa el método del padre.
¿Qué es *duck typing* en Python?	Python no revisa el tipo: si el objeto tiene el método lo llama, y si no, falla al ejecutar.

Pregunta	Respuesta
¿Cuándo usas una clase abstracta y cuándo una interfaz?	Abstracta: el padre no se debe instanciar y las hijas están relacionadas. Interfaz: un contrato para clases que no necesitan estar relacionadas.
¿Cuántas clases puedes heredar y cuántas interfaces implementar?	Una sola clase; todas las interfaces que quieras.
¿Para qué se usa UML y por qué se dice que es «agnóstico»?	Para comunicar el diseño y documentar lo que existe; es agnóstico porque no depende de la sintaxis de ningún lenguaje.
¿Qué muestra un diagrama de secuencia?	Cómo interactúan los elementos del programa, paso a paso; los mensajes se numeran para saber el orden de pedir y recibir.
¿Qué es composición?	Que una clase esté compuesta por una o varias clases (guarda objetos de otras clases).
¿Cómo obtienes el mejor pago de una lista?	Se asume que el primero `[0]` es el mejor y con un `foreach` se compara con los demás, actualizando el mejor.
¿Qué significan Arrange, Act y Assert?	Preparar datos y objetos, ejecutar lo que se prueba y verificar que el resultado sea el esperado.
¿Cuándo usas `[Fact]` y cuándo `[Theory]`?	`[Fact]` para una sola cosa; `[Theory]` para pasarle diferentes datos a la misma prueba.
¿Cómo pruebas una clase abstracta?	A través de sus clases hijas.
¿Qué es un DTO y qué no hace?	Una clase donde todo es público, que transporta datos; no valida ni decide.
¿Qué hace `[JsonIgnore]`?	Le dice a la serialización que ignore esa variable, así que no aparece en el JSON.
¿Cómo se declara un valor que puede ser número o `null`?	Con `?` en el tipo, por ejemplo `double?`.
¿En qué orden decide una cola con criterio compuesto?	Primero por el nivel de prioridad y, en empate, por el orden de llegada.